Set 17: Creating Dictionaries

Skill 17.1: Explain the purpose of a dictionary
Skill 17.2: Make a dictionary
Skill 17.3: Explain what is meant by *unhashable*
Skill 17.4: Create and populate an empty dictionary
Skill 17.5: Use the *update* method to add multiple key:value pairs
Skill 17.6: Overwrite dictionary values
Skill 17.7: Apply a *dict comprehension* to create dictionaries from lists

Skill 17.1: Explain the purpose of a dictionary

Skill 17.1 Concepts

A dictionary is an unordered set of key: value pairs.

It provides us with a way to map pieces of data to each other so that we can quickly find values that are associated with one another.

Suppose we want to store the prices of various items sold at a cafe:

- Avocado Toast is 6 dollars
- Carrot Juice is 5 dollars
- Blueberry Muffin is 2 dollars

In Python, we can create a dictionary called menu to store this data:

```
menu = {"avocado toast": 6, "carrot juice": 5, "blueberry muffin": 2}
```

Notice in the above example,

1. A dictionary begins and ends with curly braces { and }.
2. Each item consists of a key ("avocado toast") and a value (6).
3. Each key: value pair is separated by a comma.

It's considered good practice to insert a space after each comma for readability, but our code will still run without the space.

Skill 17.1 Exercise 1

Skill 17.2: Make a dictionary

Skill 17.2 Concepts

In the previous exercise, we saw a dictionary that maps strings to numbers (i.e., "avocado toast": 6). However, the keys can be numbers as well.

© Pluska

For example, if we were mapping restaurant bill subtotals to the bill total after tip, a dictionary could look like:

```
subtotal_to_total = {20: 24, 10: 12, 5: 6, 15: 18}
```

Values can be of any type. We can use a string, a number, a list, or even another dictionary as the value associated with a key!

For example,

```
students_in_classes = {"software design": ["Aaron", "Delila", "Samson"], "cartography": ["Christopher", "Juan", "Marco"], "philosophy": ["Frederica", "Manuel"]}
```

The list ["Aaron", "Delila", "Samson"], which is the value for the key "software design", represents the students in that class.

We can also mix and match key and value types. For example,

```
person = {"name": "Shuri", "age": 18, "family": ["T'Chaka", "Ramonda"]}
```

Skill 17.2 Exercise 1

Skill 17.3: Explain what is meant by *unhashable*

Skill 17.3 Concepts

We saw above that we can have a list as a value in a dictionary, but we cannot use these data types as keys. If we try to, we will get a *TypeError*,

Code	Output
`powers = {[1, 2, 4, 8, 16]: 2, [1, 3, 9, 27, 81]: 3}`	TypeError: unhashable type: 'list'

The word *unhashable* in this context means that this 'list' is an object that can be changed.

Dictionaries in Python rely on each key having a *hash value*, a specific identifier for the key. If the key can change, that hash value would not be reliable. So, the keys must always be unchangeable, hashable data types, like numbers or strings.

Skill 17.3 Exercises 1

© Pluska

Skill 17.4: Create and populate an empty dictionary

Skill 17.4 Concepts

A dictionary doesn't have to contain anything. Sometimes we need to create an empty dictionary when we plan to fill it later based on some other input.

We can create an empty dictionary like this,

```
empty_dict = {}
```

Once a dictionary is created, we can add a single key: value pair using the following syntax,

```
Dictionary[key] = value
```

Below is an example,

Code

```
menu = {}
menu["oatmeal"] = 3
menu["avocado toast"] = 6
menu["carrot juice"] = 5
menu["blueberry muffin"] = 2
print(menu)
```

Output

```
{'oatmeal': 3, 'avocado toast': 6, 'carrot juice': 5, 'blueberry muffin': 2}
```

Skill 17.4 Exercise 1

Skill 17.5: Use the *update* method to add multiple key:value pairs

Skill 17.5 Concepts

If we want to add multiple key : value pairs to a dictionary, we can use the *update()* method

Below is an example using the temperature sensors object from a previous exercise,

© Pluska

Code

```
sensors = {"living room": 21, "kitchen": 23, "bedroom": 20}
sensors.update({"pantry": 22, "guest room": 25, "patio": 34})
print(sensors)
```

Output

```
{'living room': 21, 'kitchen': 23, 'bedroom': 20, 'pantry': 22, 'guest room': 25, 'patio': 34}
```

Skill 17.5 Exercise 1

Skill 17.6: Overwrite dictionary values

Skill 17.6 Concepts

We know that we can add a key by using the following syntax,

```
menu["banana"] = 3
```

This will create a key "banana" and set its value to 3. But what if we used a key that already has an entry in the menu dictionary?

In that case, our value assignment would overwrite the existing value attached to that key. We can overwrite the value of "oatmeal" like this,

Code

```
menu = {"oatmeal": 3, "avocado toast": 6, "carrot juice": 5, "blueberry muffin": 2}
menu["oatmeal"] = 5
print(menu)
```

Output

```
{'oatmeal': 5, 'avocado toast': 6, 'carrot juice': 5, 'blueberry muffin': 2}
```

Skill 17.6 Exercise 1

© Pluska

Skill 17.7: Apply *dict comprehension* to create dictionaries from lists

Skill 17.7 Concepts

Let's say we have two lists that we want to combine into a dictionary, like a list of students and a list of their heights, in inches,

Code

```
names = ['Jenny', 'Alexus', 'Sam', 'Grace']
heights = [61, 70, 67, 64]
students = {key:value for key, value in zip(names, heights)}
```

Output

```
{'Jenny': 61, 'Alexus': 70, 'Sam': 67, 'Grace': 64}
```

Remember that *zip* combines two lists into an iterator of tuples with the list elements paired together. The *dict comprehension* above

1. Takes a pair from the iterator of tuples
2. Names the elements in the pair key (the one originally from the names list) and value (the one originally from the heights list)
3. Creates a key : value item in the students dictionary
4. Repeats steps 1-3 for the entire iterator of pairs

Skill 17.7 Exercise 1

© Pluska